Build Your Own SDR Doppler Radar

A student guide to the hardware, the software and the workflow of the project

S/LT Bilal PN, S/LT Sanaullah PN, S/LT Farhan PN, S/LT Huzaifa PN
Advisor: Assistant Professor Mohammad Mustaqim

What you will learn

- How a continuous-wave (CW) Doppler radar measures speed, and how to compute it.
- How to program a software-defined radio (PlutoSDR) from Python.
- Digital signal processing: mixing, decimation, filtering, windowing, FFT and peak detection.
- How to build an Arduino display and buzzer circuit and talk to it over USB serial.
- How to test, validate and present an engineering project.



The ADALM-PLUTO with its Tx and Rx antennas.

1. Theory in five minutes

The radar transmits a constant tone. A target moving with radial speed v reflects it with a small frequency shift fd = 2·v·fc/c. The receiver uses the same local oscillator as the transmitter, so after down-conversion only fd is left: a low audio-range tone whose frequency is proportional to speed.

Worked example: a car at 50 km/h = 13.89 m/s. fd = 2 × 13.89 × 2.482×10^{9} / 3×10^{8} = **229.8 Hz**. Measuring 229.8 Hz in the spectrum therefore means 50 km/h.

Concept	Why it matters in this project
IQ (complex) samples	The sign of the frequency tells the direction: + approaching, - receding
Tone offset (+100 kHz)	Keeps the echo away from the receiver's DC offset and LO leakage at 0 Hz
Resolution = fs / N	8000 Hz / 4096 = 1.95 Hz per bin = 0.42 km/h
Observation time	4096 samples at 8 kS/s = 0.5 s; longer = finer resolution but slower response
Cosine effect	Only the speed along the line of sight is measured; aim along the target's path
Clutter	Fans and vibration give mirror peaks at +fd and -fd; real targets give one side only

2. Parts list

Qty	Part	Notes
1	ADALM-PLUTO SDR	Firmware v0.39; use the middle (data) micro-USB port
2	2.4 GHz antennas, SMA male	Router antennas are often RP-SMA (no centre pin) and need an adapter
1	SMA extension cable, male-female, 0.5-1 m	Separates Tx and Rx antennas
1	Arduino Nano + USB cable	Clone boards use the CH340 USB chip
1	TM1637 4-digit LED module	Pins CLK, DIO, VCC (5V), GND
1	Passive buzzer (5 V)	Must be passive: an active buzzer cannot change pitch
1	NPN transistor 2N2222 or BC547	Switches the buzzer current
1	1 kΩ resistor	Base resistor
1	1N4148 or 1N4007 diode	Protects the transistor (magnetic buzzers)
1	Half-size breadboard + jumper wires	

3. Hardware build

3.1 Radar front end

- Screw one antenna onto **Tx**. Connect the extension cable to **Rx** and the second antenna to its end.
- Stand both antennas upright and about 0.5 m apart, facing the area where targets move.
- Check the connector: an SMA male antenna has a **centre pin**. A hole means RP-SMA, which screws on but makes no contact.
- Plug the Pluto into the laptop with the middle micro-USB port and wait about 30 s for the Ready LED.

3.2 Arduino, display and buzzer

Place the Nano across the centre gap of the breadboard, one pin row on each side. Connect Nano 5V to the + rail and GND to the - rail, then wire:

From	To
TM1637 CLK	Nano D2
TM1637 DIO	Nano D3
TM1637 5V / GND	+ rail / - rail
Nano D9	1 kΩ resistor → NPN base
NPN emitter	- rail (GND)
NPN collector	buzzer (-) and diode anode
buzzer (+)	+ rail (5V) and diode cathode (stripe)

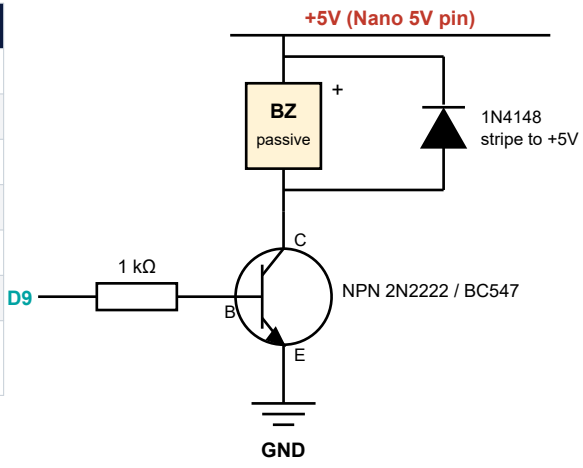


Figure 1 - Buzzer driver: D9 switches the NPN transistor; the diode absorbs the buzzer's back-EMF.

Transistor pins (flat face toward you, legs down): **2N2222 = E B C**, **BC547 = C B E**. The two types are reversed. **Before powering:** check that no wire joins + and - directly, and that 5V and GND of the display are not swapped.

4. Software setup

Step	What to install	Check
1	Python 3.10+ from python.org (tick "Add Python to PATH")	python --version
2	PlutoSDR USB driver and libiio v0.26 (in drivers/)	Pluto appears as a USB drive
3	Python packages: pip install -r requirements.txt	pyadi-iio, pylibiio, numpy, scipy, matplotlib, pyserial, sounddevice
4	Arduino IDE 2 + library "TM1637" by Avishay Orpaz	Board: Arduino Nano, Old Bootloader for clones

Project files

File	Purpose
01_pluto_test.py	Hardware check: transmits a tone and shows the received spectrum
02_doppler_radar.py	The radar: processing, detection, display, alarm, logging
03_alarm_test.py	Sends simulated passes to the Arduino (no Pluto needed)
arduino_port.py	Finds the Arduino's COM port automatically
speed_alarm/speed_alarm.ino	Arduino sketch: serial parser, LED display, buzzer PWM
START_RADAR.bat / TOOLS.bat	One-click start / menu with tests, upload and drivers

5. Code walkthrough

5.1 Configure the PlutoSDR and start transmitting

The pyadi-iio library exposes the Pluto as an object. Both oscillators are set to the same frequency, the gains are fixed, and a cyclic buffer makes the Pluto repeat the tone forever.

02_doppler_radar.py - start_pluto() and main()

```
sdr = adi.Pluto("ip:192.168.2.1")
sdr.sample_rate = 1_000_000
sdr.rx_lo = sdr.tx_lo = int(2.482e9)          # same LO for TX and RX
sdr.gain_control_mode_chan0 = "manual"
sdr.rx_hardwaregain_chan0 = 30                # dB, lower if CLIPPING
sdr.tx_hardwaregain_chan0 = -10               # dB attenuation
sdr.rx_buffer_size = 64000                    # 64 ms per read

n = np.arange(64000)                          # 6400 whole periods of 100 kHz
sdr.tx_cyclic_buffer = True
sdr.tx(0.5 * np.exp(2j * np.pi * 100e3 * n / 1e6) * 2**14)
```

Tip: the buffer holds a whole number of tone periods, so the repeated tone has no phase jump.

5.2 Down-conversion and decimation

Multiplying by a complex exponential shifts the 100 kHz tone to 0 Hz. Averaging blocks of 25 samples is a cheap low-pass filter whose nulls fall on multiples of 40 kHz; an FIR filter then removes everything above 3.5 kHz before keeping every 5th sample.

02_doppler_radar.py - Downconverter

```
class Downconverter:
    def __init__(self):
        self.mixer = np.exp(-2j * np.pi * TONE_HZ * np.arange(RX_BLOCK) / SAMPLE_RATE)
        self.h2 = dsp.firwin(101, 3.5e3, fs=SAMPLE_RATE / DECIM_1)
        self.z2 = np.zeros(len(self.h2) - 1, complex)      # filter state between blocks

    def __call__(self, x):
        x = x * self.mixer                                 # tone -> 0 Hz
        y = x.reshape(-1, DECIM_1).mean(axis=1)            # 1 MS/s -> 40 kS/s
        y, self.z2 = dsp.lfilter(self.h2, 1, y, zi=self.z2)
        return y[::DECIM_2]                                # 40 kS/s -> 8 kS/s
```

5.3 Spectrum and detection

The newest 512 samples are appended to a 4096-sample history (0.5 s). The mean is removed (direct TX-to-RX leakage is a constant), a Hann window reduces spectral leakage, and the FFT gives the spectrum. A target is accepted only if it stands out from the noise and from its mirror image.

02_doppler_radar.py - run_radar()

```
history = np.concatenate([history[len(y):], y])
x = history - history.mean()                        # remove TX leakage (DC)
spec = np.fft.fftshift(np.fft.fft(x * window)) / NFFT
p_db = 20 * np.log10(np.abs(spec) + 1e-15)

noise_db = np.median(p_db[band])                    # noise floor
k = np.flatnonzero(band)[np.argmax(p_db[band])]     # strongest bin
snr = p_db[k] - noise_db
km = NFFT - k                                       # mirror bin (-fd)
mirror_db = p_db[max(km - 2, 0):km + 3].max()
hit = snr >= DETECT_DB and p_db[k] - mirror_db >= MIRROR_DB
hits = hits + 1 if hit else 0                       # confirm over 2 frames
```

5.4 Sub-bin precision

A parabola through the peak bin and its two neighbours finds the true peak between bins. On a simulated target this reduced the error from up to 0.2 km/h to below 0.01 km/h.

02_doppler_radar.py

```
def peak_offset(p_db, k):
    a, b, c = p_db[k - 1], p_db[k], p_db[k + 1]
    curve = a - 2 * b + c
    return float(np.clip(0.5 * (a - c) / curve, -0.5, 0.5)) if curve < 0 else 0.0

fd = freqs[k] + peak_offset(p_db, k) * FS_DOPPLER / NFFT
kmh = fd * C / (2 * CENTER_FREQ) * 3.6
```

5.5 Pass tracking and capture

While a target is present the program keeps the highest speed of the pass. After about 1 s without a target the pass ends: its peak speed is held on the display, written to speed_log.csv, and captured if it is above the speed limit.

02_doppler_radar.py

```
if proximity > 0:                           # target present in this frame
    if abs(kmh) > abs(pass_peak):
        pass_peak = kmh                     # keep the peak speed of the pass
    pass_frames += 1
    lost_frames = 0
elif pass_frames:
    lost_frames += 1
    if lost_frames >= PASS_END_FRAMES:      # ~1 s without target
        if pass_frames >= PASS_MIN_FRAMES:
            end_pass(pass_peak)             # hold, log, capture if > limit
        pass_peak, pass_frames, lost_frames = 0.0, 0, 0
```

5.6 Laptop → Arduino protocol

Every frame (64 ms) one text line is sent at 115200 baud: `S45.3,P70,V75,L38.0,F0` = live speed, proximity %, volume %, last speed (-1 = none) and the capture flag. If no line arrives for 0.5 s the Arduino goes silent.

5.7 Arduino sketch

The buzzer uses Timer1 in fast-PWM mode on pin D9: the timer's TOP value sets the pitch and the compare value sets the duty cycle (loudness). The beep pattern comes from millis().

speed_alarm.ino

```
void buzzerOn(float pitchHz, int vol) {
  uint16_t top = (uint16_t)(2000000.0 / pitchHz) - 1;   // 16 MHz / prescaler 8
  float duty = 0.02 + 0.48 * vol / 100.0;               // 2 % .. 50 %
  ICR1 = top;
  OCR1A = (uint16_t)(top * duty);
  TCCR1A = _BV(COM1A1) | _BV(WGM11);                    // fast PWM, TOP = ICR1
  TCCR1B = _BV(WGM13) | _BV(WGM12) | _BV(CS11);
}

float beepsPerSec = 1.0 + 9.0 * proximity / 100.0;      // closer = faster beeps
unsigned long period = (unsigned long)(1000.0 / beepsPerSec);
bool beepOn = (millis() % period) < period / 2;
float pitch = 600.0 + 6.0 * volume;                     // 600 .. 1200 Hz
```

5.8 Safe shutdown

A radar must never keep transmitting after the program ends. All work runs inside `try ... finally`; on any exit (window closed, Ctrl+C, error) the TX buffer is destroyed and the TX attenuation is set to -89 dB.

6. Build and test workflow

Build in stages and test each stage before adding the next. When something fails you then know which stage caused it.

Stage	Action	Pass criterion
1	Install Python, drivers and packages	python -c "import adi" runs without error
2	Connect Pluto and antennas; run 01_pluto_test.py	Strong peak at +100 kHz, below 0 dBFS
3	Run the radar on the laptop only; wave a hand	Speed shown, direction sign correct
4	Wire the display; upload speed_alarm.ino	Display shows 88:88, then ----
5	Run 03_alarm_test.py	Display shows 15, 35, then 72 blinking
6	Add the buzzer circuit	Start-up chirp; beeps during the alarm test
7	Full system: START_RADAR.bat	Walking and vehicle passes shown, held and logged
8	Validation (section 7)	Error table filled in for the report

7. Testing and validation

- **Reference speed:** mark 10 m on the ground and time each pass with a stopwatch; reference = distance / time. For vehicles, use the speedometer at a constant speed.
- Walk or drive **straight toward the antennas** to avoid the cosine error.
- Repeat 5 times per speed (slow walk, fast walk, jog, vehicle) and compare with speed_log.csv.
- Run 5 minutes in an empty room to count false alarms. Fans and moving objects must be out of view.
- If CLIPPING appears, increase the antenna separation or lower --rx-gain.

Run	Distance (m)	Time (s)	Reference (km/h)	Radar (km/h)	Error (%)
1	10				
2	10				
3	10				

8. Troubleshooting

Symptom	Cause and fix
PlutoSDR not found / Open unlocked: -16	Wrong USB port, drivers missing, or another radar window still open
No peak in 01_pluto_test.py	RP-SMA antenna on an SMA port (no contact): use an adapter
CLIPPING on screen	Antennas too close or RX gain too high
Targets with nobody moving	Ceiling fan or vibrating objects in view
Upload error: not in sync	Clone Nano: choose ATmega328P (Old Bootloader)
Access is denied on the COM port	Arduino IDE or Serial Monitor still open
Display blank	CLK/DIO swapped or loose, or 5V/GND not connected
Buzzer silent	Transistor pins reversed (2N2222 vs BC547) or an active buzzer used

9. Presenting the project

Suggested slide order

- Problem and objective
- Doppler principle with the worked example
- System architecture (hardware block diagram)
- Signal-processing chain
- Speed-camera logic, display and buzzer
- Results: validation table and screenshots
- Live demonstration
- Conclusion and future work

Demonstration script

- Show the hardware and explain the Tx/Rx antennas.
- Start START_RADAR.bat and explain the live spectrum.
- Walk toward the antennas: point out the live speed, the beeps and the held last speed.
- Do a fast pass over the limit: show the red CAPTURED message and the blinking LED.
- Open speed_log.csv to show the recorded passes.

Questions to prepare for

Question	Short answer
Why a tone offset of 100 kHz?	The receiver has a DC offset and LO leakage at 0 Hz; the offset keeps the echo away from it
What limits the resolution?	Observation time: 0.5 s gives 1.95 Hz = 0.42 km/h per bin
What is the maximum speed?	Set by the 1 kHz search limit: about 218 km/h
How are fans rejected?	A vibrating object gives peaks at +fd and -fd; the mirror test rejects it
Can it measure distance?	No, CW radar measures speed only; FMCW would add range
Why does an off-axis target read low?	Only the radial component v·cos(angle) shifts the frequency

10. Exercises

- Change NFFT to 2048 and 8192: how do resolution and response time change?
- Plot speed against time for a walking pass and identify arm and leg motion.
- Add a webcam snapshot when a pass is captured.
- Replace the dipoles with directional patch antennas and measure the range gain.